Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Lin Xie	linxie@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

Steganography or Flashcard Study Tool

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

There are websites for steganography decoding that exist online. Some of these include StegOnline, StegoToolkit, and Aperi'Solve.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

The purpose would be to hide messages in images. I would want this application to be used to communicate secretly.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The ultimate goal would be to take in user input of an image and a message (string), process the data using algorithms and functions, and output the image but with the message hidden within.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

A message to be hidden.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

Using if statements so that not all code has to be used and iterating with loops for repetition areas.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The image but with the message hidden within.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Cover Image, Secret Message, Pixel, Output Image

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Pixel Array, Image Dimensions, Least Significant Bits (LSBs), Payload Bit Length

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Concepts that I don't really understand yet and steganography having concepts that are different from the usual patterns I'm used to from class.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

Doing your own separate research on things you're confused on and not solely relying on what LLMs say.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

Further explore a concept that I'm interested in and learn more.

Part 5: Research & Brainstorming Conversation

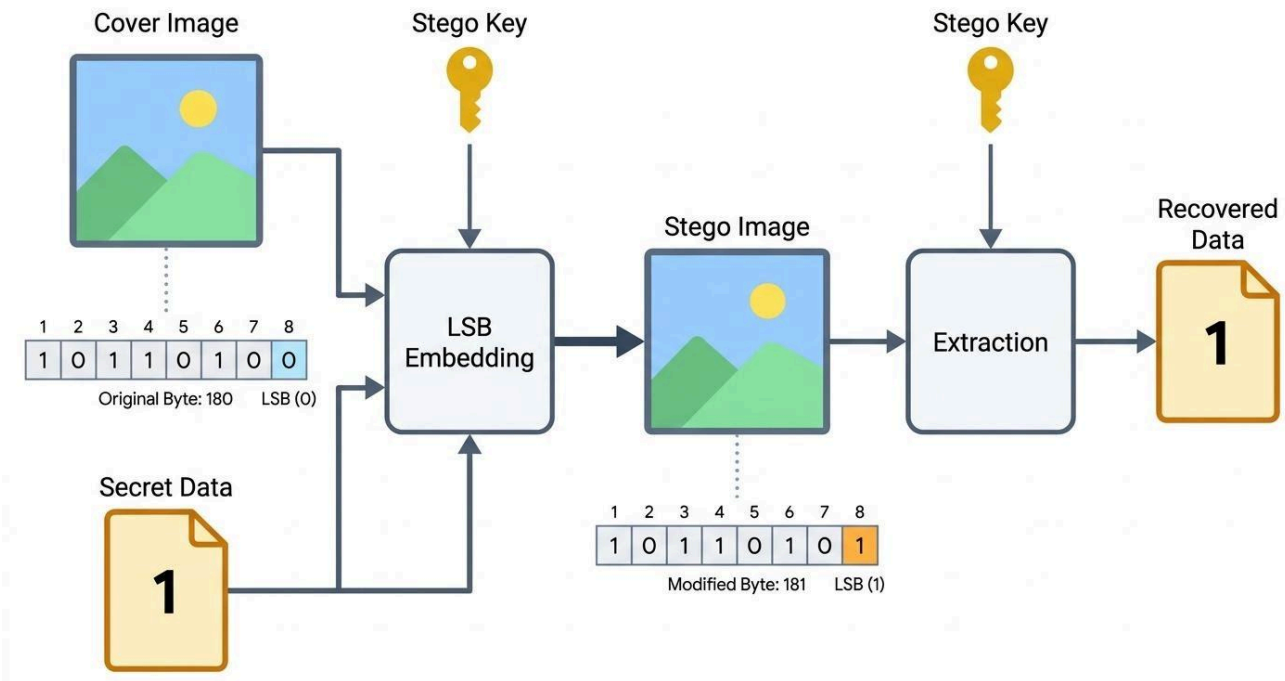
Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a simple steganography tool that hides secret text inside image files. What are 3 simple, core features this tool should have that can also be beginner friendly?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

Three core features make a steganography tool easy to build while remaining practical:



LSB encoding modifies only tiny pixel values. Source: Made with Google AI

1. Least Significant Bit (LSB) Encoding & Extraction

The backbone of basic image steganography is modifying the lowest bit of each pixel's color

channel (RGB).

- **Why it works:** Changing the binary value of a color channel's least significant bit (e.g., from `11111110` to `11111111`) alters the pixel color by a fraction of a percent—completely invisible to the human eye.
- **Beginner implementation:** Convert your secret text into a stream of binary bits (`0`s and `1`s), loop through the image's RGB pixels, and swap each pixel's last bit with a bit from your secret message.

2. Message Length Header or End-of-Message Marker

When extracting a hidden message from pixel bits, your program needs to know where the secret text stops so it doesn't read garbage binary data from the remaining pixels.

- **Header Approach:** Embed a fixed 32-bit integer at the very beginning of the image indicating the length of the secret text (e.g., "message is 45 characters long").
- **Delimiter Approach:** Append a unique string of characters to the end of your secret message (e.g., `###END###`). During extraction, read bits until that exact pattern is encountered.

3. Automatic Lossless Format Enforcement (PNG/BMP Output)

Lossy compression algorithms (like JPEG) discard fine pixel data to shrink file sizes, which destroys hidden LSB data instantly.

- **Why it's essential:** Beginners often get confused when an image saved as `.jpg` fails to decrypt.
- **Implementation:** Force your tool to read and export images exclusively in lossless formats such as **PNG** or **BMP**, or automatically convert incoming files to PNG upon saving.

Basic Implementation Concept (Python/Pillow)

Python

```
None
# Concept for LSB encoding a single bit into a color channel
def hide_bit(color_channel_byte, secret_bit):
    # Clear the last bit, then bitwise OR with the secret bit (0
or 1)
    return (color_channel_byte & ~1) | secret_bit

# Concept for extracting that single bit
def extract_bit(color_channel_byte):
```

```
    return color_channel_byte & 1
```

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

Is there a simpler alternative for Message Length Header or End-of-Message Marker and Automatic Lossless Format Enforcement? It sounds a little complex for a beginner but can you explain it still?

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

From this conversation, I have learned a little more about the specifics involved in steganography and have an outline of how to build this program.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will refer back to previous course material, get help from TAs and professors, and use online resources.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I would first check for any syntax errors, and then reread my code and try to find where the cause of error(s) are. If that doesn't work, then using the method of commenting out a line or multiple lines to test where the code runs until to narrow down the lines of code that I need to review.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

1. Identifying Properties: For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?

- Details I need: (e.g., Ingredient name, weight, cost)
- Details I will ignore: (e.g., Ingredient color, brand, expiration date)

Here is how to structure this answer for your assignment, focusing on the most important entity: the `Cover Image` (or `Pixel Data`).

Most Important Entity: `Cover Image` / `Pixel Data`

Details I need:

Pixel Array / Byte Buffer: The raw collection of Red, Green, and Blue (RGB) integer values (0–255) where the payload bits will be stored.
Image Dimensions (Width & Height): To calculate the total pixel capacity and ensure the image is large enough to hold the payload.
Least Significant Bits (LSBs): The specific low-order bit positions of each color byte that will be targeted for replacement.
Payload Bit Length: The exact size of the secret message (in bits) to know when to stop reading or writing data.

Details I will ignore:

EXIF Metadata: Camera settings, GPS coordinates, date taken, and equipment details embedded in the file header.
Visual Content & Subject Matter: What the image actually depicts

(e.g., a dog, a sunset, or a building).
File Compression History & Original Filename: The creator, original file creation timestamp, or host directory structure.

explain each of the details and what they are and how they are involved in steganography

Here is a detailed breakdown of each property needed for steganography, explaining what it is and how it functions during the hiding and extraction processes.

1. Pixel Array / Byte Buffer

What it is: A structured collection of memory storing the color values for every point on the screen. In a standard 24-bit RGB image, each pixel consists of three separate byte channels: **Red**, **Green**, and **Blue**. Each byte holds an integer value from `0` to `255` (written in binary as `00000000` to `11111111`).
Role in Steganography: The byte buffer serves as the storage grid where the secret message lives. The program iterates through these bytes one by one, replacing the original lowest-order bits with bits from the secret message.

2. Image Dimensions (Width & Height)

What it is: The pixel measurements of the image grid (e.g., 1920×1080).
Role in Steganography: Dimensions determine the total **storage capacity** of your cover image.

Total Pixels = $\text{Width} \times \text{Height}$

Total Color Bytes = $\text{Pixels} \times 3 \text{ (Red, Green, Blue)}$

If an image is
800×600, it contains
$1,440,000$ color bytes. Storing 1 bit per byte yields a maximum payload capacity of
$1,440,000$ bits (
180 KB). The program uses the dimensions to verify whether the secret message will physically fit before writing any data.

3. Least Significant Bits (LSBs)

What it is: The rightmost bit (the 1s place) in a binary byte. Changing the Most Significant Bit (the leftmost bit, or 128s place) drastically alters the byte's value. Changing the **Least Significant Bit** alters the value by at most
± 1.

Example: Changing `11111111` (`255`) to `11111110` (`254`) results in a shift that is imperceptible to the human eye.

Role in Steganography: The LSB is the exact bit position altered during the embedding process. Using bitwise operations (`AND` masks to clear the bit, `OR` masks to set it), the program overwrites only this specific position to store secret payload bits without visibly altering the host image.

4. Payload Bit Length

What it is: An integer representing the total count of bits contained in the secret message.

Role in Steganography: The computer treats memory as a continuous stream of data. Without knowing the payload length, an extraction script would not know when to stop reading pixel bits and would keep reading random image data as gibberish text. To resolve this:

During Embedding: The program often embeds the **payload size** into the first 32 pixels of the image before storing the actual message.

During Extraction: The decoder reads the first 32 LSBs to extract the size integer, telling the program exactly how many subsequent bits to read before stopping.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.

Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I expect mainly datclasses, routes, state, nested for loops, and if statements. I want to improve my understanding of nested for loops and the mechanics of LSB steganography in general.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect the easiest part of the project to be setting up the page layouts like Button and Text because they are pretty straightforward and easy to visualize. I expect the hardest part to be the actual encoding and decoding of messages into images because that is a concept that is new to me, but I think it will be a valuable challenge.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

As mentioned before, I believe understanding and implementing the encoding and decoding aspects of steganography will be the tasks that require the most time and effort because of the unfamiliarity from working with concepts that are still relatively new to me. Core dataclasses such as State and are components that should be implemented first because it creates the foundation for the rest of the program to base itself off of, as well as mapping the layout of each page and which buttons lead to new pages.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Due to the fact that this is a big program, it is unlikely that I would finish it in one sitting, so ensuring regular dedication to work on the program is crucial for success. To do this, I would plan out periods throughout the week that are designated to working on my project. Another possible interference of my work is the common distraction of being on my phone, which I can minimize by turning on Do Not Disturb and keeping it away from arm's reach while I am working so that I won't feel tempted to

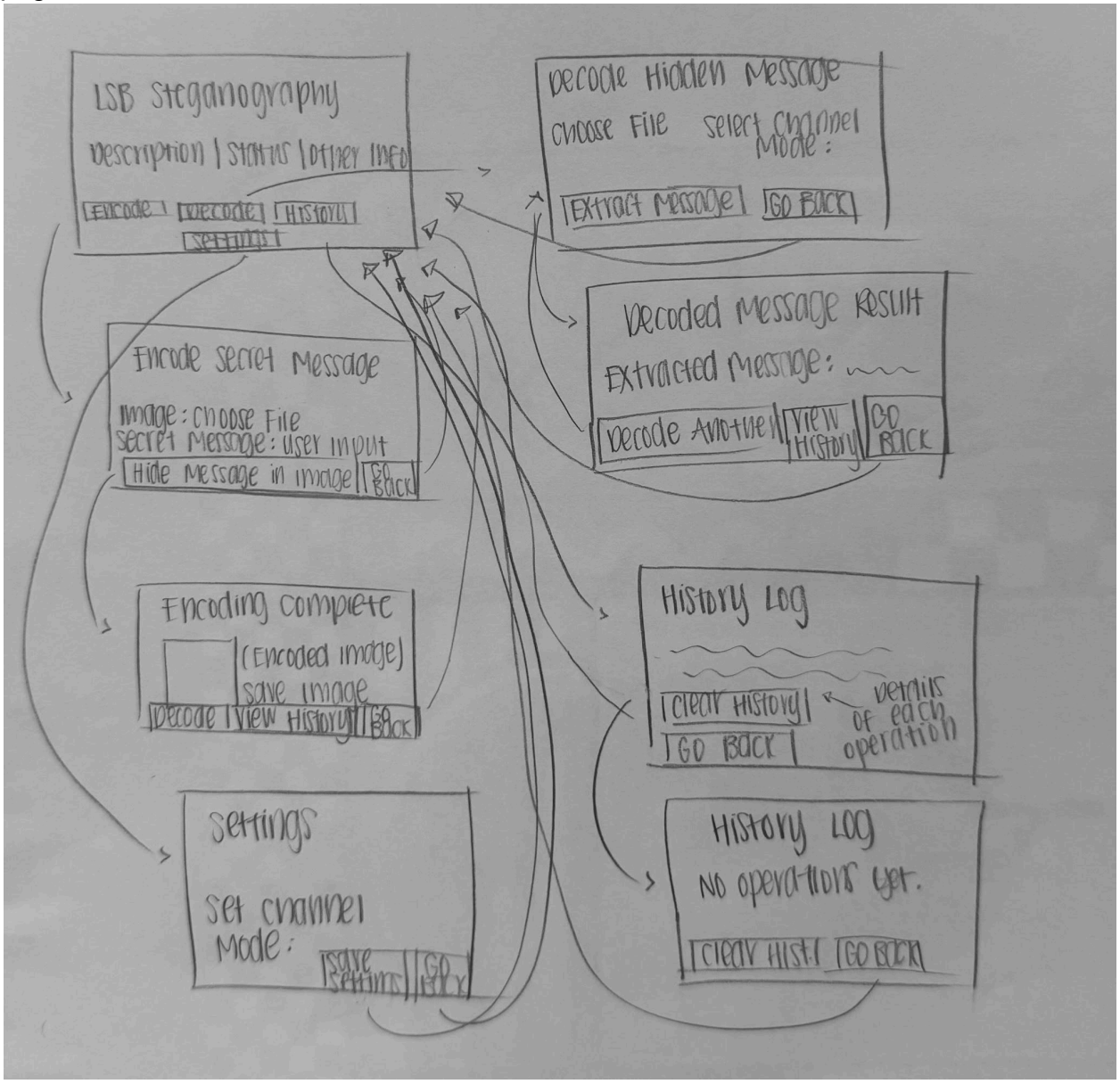
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.



2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**A web-based Least Significant Bit (LSB) steganography studio that allows users to hide secret text messages inside PNG or JPG images and extract hidden messages from encoded PNG files. It also tracks session encoding/decoding activity and lets users configure RGB or Red-only bit-embedding modes.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	**history: List[StegoRecord] → Stores a chronological list of StegoRecord objects** ● **Meaningfully modified in process_encode and process_decode by appending a new StegoRecord instance after each successful operation and resets back to empty list when user clicks "Clear History"** **last_encoded_b64: str → Stores the Base 64 encoded PNG data URI of the most recent encoded image** ● **Updated in process_encode and reset in clear_history** **last_decoded_message: str → Stores the most recent extracted secret text** ● **Resets to "" if no message is found or when clear_history is called** **status _message: str → Displays live feedback on the home dashboard** ● **Updated across process_decode, clear_history, and save_settings** **preferred_channel: str → Sets to either "RBG" or "Red Only" when process_encode or process_decode is ran** ● **Modified in save_settings and used as the**

	default in encode_page and decode_page
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	**`operation: str → Either “Encode” or “Decode”`** **`filename: str → Set to "stego_output.png" in process_encode or "uploaded_stego.png” in process_decode`** **`message_preview: str → Stores a shortened snippet of the hidden or decoded message (up to 30 characters)`** **`channel_mode: str → “RGB” or “Red Only”`**
Routes (minimum 7) The names of your route functions	**index** **encode_page** **process_encode** **decode_page** **process_decode** **history_page** **clear_history** **settings_page** **save_settings**
Pages (minimum 5) A brief description of the pages in your site	**index: Homepage that shows description of program, current status, current channel mode, total operations, and buttons to navigate to different pages (encode, decode, history, settings)** **encode_page: Allows users to upload an image and enter message to be hidden, as well as the option to select either “RBG” or “Red Only”** **process_encode: Displays the newly generated PNG image with their chosen message hidden that the user can save, along with the channel mode used and total characters hidden** **decode_page: Allows users to upload an encoded PNG image, choose the channel mode used during encoding, and enter an optional**

	prefix filter **process_decode: Displays the secret text hidden within the uploaded image or shows that no hidden message was found or if the prefix did not match** **history_page: Lists all encoding and decoding operations and a summary of each, as well as a button to clear history** **settings_page: Explains how LSB steganography works and lets the user change the default channel mode**
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	**FileUpload: Lets users upload images** **TextArea: Lets users enter multi-line text to embed into an image** **SelectBox: Lets users choose between "RBG" and "Red Only"** **TextBox: Lets users enter an optional passphrase prefix to validate the decoded message**
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	• **If the image uploaded by the user is in a compatible format to the program or if nothing is uploaded at all** **→ Prevents the program from trying to read files when there are none or running into errors if it is not a compatible format** • **If the message entered by the user is able to be hidden within the chosen picture** **→ Also prevents the program from trying to read and utilize the message when there is none entered** • **If any message was found within uploaded picture while decoding** **→ If a message is found, then it will be added to the history of operations. But if a message isn't**

	found, then an error message will be displayed
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	**A loop that goes through all the instances of each operation and gets its attributes to display in the history. This loop is meaningful to the site because it allows the user to view a log of all their encoding or decoding activity in case they want to look back at their past history rather than being restricted to only their most recent action.**

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

how are each of my state dataclass attributes meaningfully modified throughout the site

Each of the 5 attributes in your `State` dataclass is modified across specific routes to track user activity, cache results, and store preferences:

steganography.py

```
history: List[StegoRecord]
```

steganography.py

Modified in `process_encode` **&** `process_decode`: Appends a

new `StegoRecord` dataclass instance (recording the operation type, filename, truncated message preview, and channel mode) whenever an image is successfully encoded or a valid hidden message is decoded.

steganography.py

Modified in `clear_history`: Reset back to an empty list (`[]`) when the user clicks "Clear History".

steganography.py

Used in `index` **&** `history_page`: Read on the home dashboard to display the total operation count and looped over on the history page to render the numbered activity log.

steganography.py

`last_encoded_b64: str`

steganography.py

Modified in `process_encode`: Updated with the Base64 PNG data URI (`data_url`) of the newly generated stego image right after encoding succeeds.

steganography.py

Modified in `clear_history`: Cleared back to an empty string (`""`) when the user resets their session history.

steganography.py

`last_decoded_message: str`

steganography.py

Modified in `process_decode`: Updated with the full `extracted` secret string if a hidden message is found, or reset to `""` if no valid LSB message is detected in the uploaded image.

steganography.py

Modified in `clear_history`: Cleared back to `""` when the user clears their session history.

steganography.py

`status_message: str`

steganography.py

Modified in `process_encode` **&** `process_decode`: Updated with real-time success confirmations (e.g., number of characters encoded/decoded) or specific validation/exception error messages (e.g., missing file, message too long for image capacity, or no hidden message found).

steganography.py

- **Modified in** `clear_history` **&** `save_settings`: Updated to confirm that history was cleared or that the default channel mode was saved.
  steganography.py
- **Used in** `index`: Displayed directly on the main dashboard so the user sees live feedback on their most recent action.
  steganography.py

`preferred_channel: str`
steganography.py

- **Modified in** `save_settings`: Updated from `"RGB"` to `"Red Only"` (or vice versa) when the user saves their setting on the Settings page.
  steganography.py
- **Used in** `index`**,** `encode_page`**,** `decode_page`**, &** `settings_page`: Displayed on the dashboard and passed as the `default_value` into the `SelectBox` dropdowns so the user's preferred mode is pre-selected across the app.
  steganography.py

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.